

#### Your Code

```
1 POSITIVE_KEYWORDS = ["good", "great", "excellent", "awesome", "love"  
2 NEGATIVE_KEYWORDS = ["bad", "poor", "terrible", "worst", "slow", "di  
3  
4 def preprocess_text(text):  
5     text = text.lower()  
6     text = text.strip()  
7     punctuation_list = [".", ",", "!", "?", ";", ":", "-", "_", "(",  
8     for char in punctuation_list:  
9         text = text.replace(char, "")  
10    return text  
11  
12 def analyze_sentiment(cleaned_text):  
13     words = cleaned_text.split()  
14     positive_count = 0  
15     negative_count = 0  
16  
17     for word in words:  
18         if word in POSITIVE_KEYWORDS:  
19             positive_count += 1  
20         elif word in NEGATIVE_KEYWORDS:
```

#### Input

```
3  
    GREAT product! I love the FAST  
    delivery.  
    The item was terrible, very SLOW
```

#### Output

```
Feedback #1: great product i  
love the fast delivery  
Positive Words: 3, Negative  
Words: 0  
Sentiment: Positive
```

#### Your Code

```
1 def analyze_password(password):  
2     if not password:  
3         return "Weak", ["Password cannot be empty."]  
4     if len(password) > 64:  
5         return "Weak", ["Password exceeds maximum length limit."]  
6  
7     has_upper = False  
8     has_lower = False  
9     has_digit = False  
10    has_symbol = False  
11    symbols = "!@#$%^&*()_+-=[]{}|;:\'", "<>?/~`"  
12  
13    for char in password:  
14        if 'A' <= char <= 'Z':  
15            has_upper = True  
16        elif 'a' <= char <= 'z':  
17            has_lower = True  
18        elif '0' <= char <= '9':  
19            has_digit = True  
20        elif char in symbols:
```

#### Input

```
P@ssw0rd2026
```

#### Output

#### Your Code

```
1 def extract_errors(log_lines):
2     error_logs = []
3     for line in log_lines:
4         line_clean = line.strip()
5         if line_clean.startswith("ERROR") or "ERROR" in line_clean:
6             error_logs.append(line_clean)
7     return error_logs
8
9 def categorize_and_count(error_logs):
10    error_counts = {}
11    unique_errors = []
12
13    for log in error_logs:
14        if "Database" in log or "DB" in log:
15            category = "Database Error"
16        elif "Network" in log or "Connection" in log or "Timeout" in log:
17            category = "Network Error"
18        elif "Auth" in log or "Permission" in log or "Denied" in log:
19            category = "Authentication Error"
```

#### Input

```
6
INFO: System started successfully
ERROR: Database connection failed
WARNING: High memory usage
```

#### Output

```
--- ERROR SUMMARY ---
Total Errors Found: 4

Error Count by Category:
  Database Error: 2
  Network Error: 1
```

#### Your Code

```
1 def extract_domain(email):
2     email = email.strip()
3     if "@" not in email:
4         return None
5
6     parts = email.split("@")
7     if len(parts) != 2:
8         return None
9
10    username, domain = parts[0], parts[1]
11
12    if not username or not domain or "." not in domain:
13        return None
14
15    return domain.lower()
16
17 def analyze_domains(email_list):
18     domain_counts = {}
19     invalid_count = 0
20
21     for email in email_list:
22         domain = extract_domain(email)
```

#### Input

```
5
user1@gmail.com
user2@company.org
invalid-email.com
```

#### Output

```
Domain Frequency Analysis:
  gmail.com: 2
  company.org: 2

Most Common Domain: gmail.com
(Frequency: 2)
```